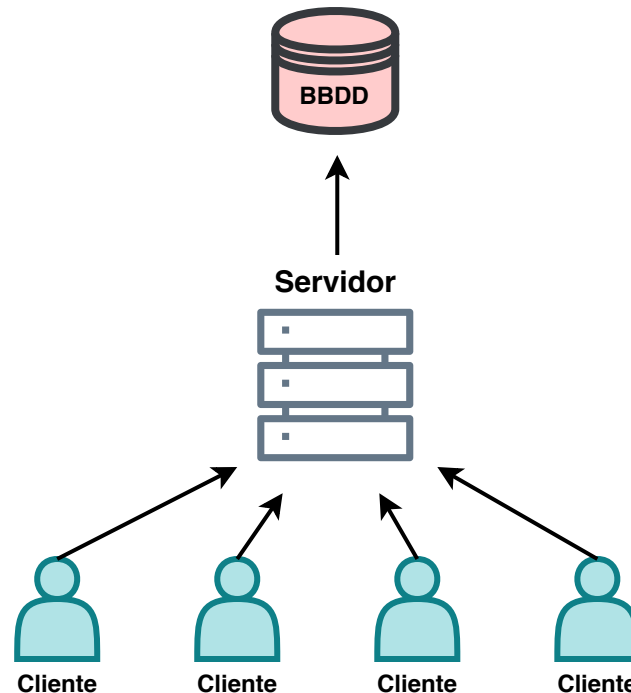


SQL CON SQLITE

CONCEPTOS BÁSICOS DE BASES DE DATOS

¿QUÉ ES UNA BASE DE DATOS?

Una base de datos es un conjunto de datos que se almacenan y organizan de forma estructurada para que sean fácilmente accesibles y gestionables.



TIPOS DE BASES DE DATOS

1. **SQL** (Structured Query Language) o Relacionales
2. **NoSQL** o No relacionales

SQL (Structured Query Language) o Relacionales

- Son las bases de datos estándar, **las de toda la vida**.
- Sirven sobre todo para cuando tenemos datos que **se relacionan entre tablas**.
- Cuando todos los registros vayan a tener los **mismos tipos de datos**.
- Cuando necesitemos **relaciones** entre unas tablas y otras.
- La estructura es mucho más **rígida**.

NoSQL o No relacionales

- Están diseñadas para hacer peticiones **más rápidas**.
- Tienen **menos restricciones**.
- Cuando simplemente necesitemos guardar datos **sin necesidad de relacionar entre unos datos y otros**.
- Son más escalables y se adaptan mejor a **datos no estructurados**.
- Unos registros y otros pueden tener **distintos datos guardados**.
- Big Data

MODELOS DE DATOS

👉 **Tabla** → La que contiene toda la información de cada elemento, todos los registros.

→ *tareas*

👉 **Registros** → Es una colección de datos referentes a un tema. (Cada renglón o fila de la tabla)

→ | 1 | Pasear a las perras | Completado |

👉 **Campos/Columnas** → Es la unidad básica de información (cada columna).

→ *INTEGER, TEXT, REAL, BLOB, NULL*

👉 **Dato** → Es el valor de cada una de las celdas, la información que nos interesa guardar.

→ *"Pasear a las perras"*

TIPOS DE DATOS

LO QUE PODEMOS GUARDAR EN CUALQUIER CAMPO DE UNA TABLA.

TIPO	DESCRIPCIÓN
TEXT	Cualquier tipo de texto.
INTEGER	Números enteros sin coma.
REAL	Números decimales.
NULL	El valor <i>null</i> , es decir, nada
BLOB	Datos binarios, es decir archivos como fotos
NUMERIC	Acepta tanto enteros como decimales pero deja a SQLite que establezca el tipo ¡ojo con este!

TIPOS DE DATOS

¿PUEDO USAR OTROS TIPOS?

Sí, podemos usar otros tipos de datos, pero SQLite los convierte a su tipo real. Por ejemplo, si declaramos un campo como `VARCHAR(255)` lo convertirá a `TEXT`.

Tipo declarado	Afinidad real
<code>VARCHAR(255)</code>	<code>TEXT</code>
<code>DECIMAL</code>	<code>NUMERIC</code>
<code>BOOLEAN</code>	<code>NUMERIC</code>
<code>DATETIME</code>	<code>NUMERIC</code>
<code>FLOAT</code>	<code>REAL</code>
<code>INT</code>	<code>INTEGER</code>
<code>TINYINT</code>	<code>INTEGER</code>

¿CÓMO TRABAJAMOS CON LAS BASES DE DATOS DE SQLITE?

A diferencia de otras bases de datos, SQLite **no tiene un servidor al que conectarse**. En su lugar, se trabaja directamente con **un archivo** de base de datos. Este archivo puede estar en el disco duro o guardado en la nube.

→ *tareas.db*

¿CÓMO VEMOS NUESTRAS BASES DE DATOS?

Podemos ver nuestras bases de datos usando varios programas, pero el más sencillo es **DB Browser for SQLite**. Este programa nos permite ver y editar nuestras bases de datos de forma sencilla y visual.

También podemos usar VSCode con alguna extensión. Una muy buena es **DBCode**.

ANTES QUE NADA, HAY QUE DISEÑAR NUESTRA BASE DE DATOS

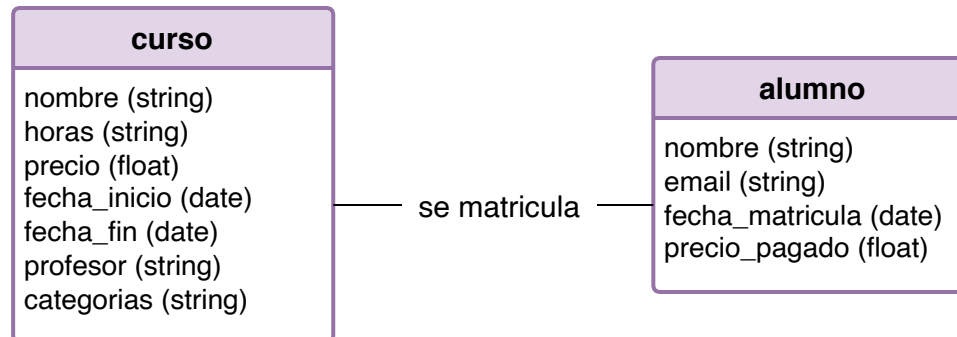
Lo primero y más importante es entender **cómo se va a organizar cada una de nuestras bases de datos**. Para ello solemos hacer una serie de pasos en los que veremos todas las necesidades que tendrá nuestra base de datos.

1. MODELO CONCEPTUAL - ENTENDER LOS REQUERIMIENTOS

MODELO DE NEGOCIO

Lo primero que hay que hacer es **organizar** bien nuestro **modelo de negocio**, es decir, **cómo funciona nuestra empresa** y qué **tipos de datos** vamos a querer guardar y de qué forma.

Aquí podemos hacer algún mapa conceptual de lo que pueda contener la BBDD.



Lo pasamos a excel para ver cómo se verían nuestros datos una vez añadidos los registros.

CURSO						
nombre	horas	precio	fecha_inicio	fecha_fin	profesor	categorias
HTML	30	50	2023-05-15	2023-05-30	Ivan	programacion, codigo, frontend
CSS	50	100	2023-05-30	2023-06-15	Ivan	programacion, css, frontend
JS	100	200	2023-06-15	2023-07-09	Samuel	programacion, avanzado, frontend

ALUMNO			
nombre	email	fecha_matricula	precio_pagado
Alex Pacheco	alex@pacheco.com	2023-03-29	50
Juan Pérez	juan@perez.com	2023-03-08	40

2. MODELO LÓGICO:

- Ahora hay que sacar del modelo conceptual toda la información que vamos a necesitar para crear nuestra base de datos.
 - Entidades/Tablas
 - Atributos/Columnas
 - Llaves/Primary Key, Foreign Key, Unique
 - Relaciones entre tablas

ENTIDAD / TABLA

Se considera entidad a cualquier elemento de nuestro modelo de negocio 👉 nombre, persona, lugar, cosa, evento, etc.

Cada entidad se convertirá en **una tabla** dentro de nuestra BBDD.

ATRIBUTOS / COLUMNAS

Cada uno de los datos que tiene una entidad.

Ej: nombre, email, fecha_compra, telefono, edad, esta_activo, etc.

LLAVES

Son identificadores únicos para cada columna. Nos ayudarán a relacionar las tablas entre sí.

1. **Primary Key** → Identificador único que tiene cada registro.
2. **Foreign Key** → Son las que van a permitir la relación entre una tabla y otra.
3. **Unique** → No sería una llave al uso, pero sí que es un tipo de dato que va a ser único en toda su tabla.

RELACIONES

Son ese tipo de asociación que hay entre las tablas.

1.	1 a 1 (1 to 1) 	empleado $\leftrightarrow$ DNI
2.	1 a Muchos (1 to N) 	pais $\Rightarrow$ ciudad
3.	Muchos a Muchos (N to M) 	alumno $\Leftrightarrow$ curso

Por último:

Al saber las relaciones que hay entre las tablas, podemos empezar a darnos cuenta de que va a haber distintos tipos de tablas. Yo lo divido en **3 tipos de tablas**:

1. TABLA DE DATOS:

La que guardará todos los datos que nos interesan.

*Una tabla **profesor** tendrá una tabla y guardará su:*

nombre	apellidos	email	etc.
Ivan	Luengo	ivluengo@gmail.com	–

2. TABLA DE CATÁLOGO:

Guardará un listado de “tipos” y tendrá un número determinado de datos que usualmente no varía en el tiempo. Esta entidad la relacionaremos con una tabla de datos.

Es la típica tabla “métodos de pago” o “tipo de usuario” en la que guardaremos los 3 o 4 tipos que hay.

id	metodo_de_pago
1	tarjeta
2	transferencia
3	efectivo

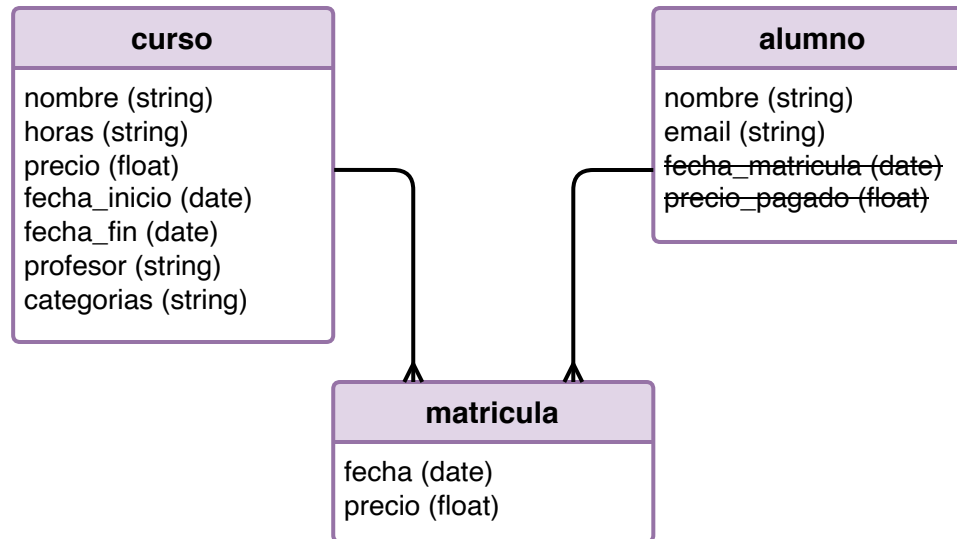
3. TABLA DE TIPO ENLACE/PIVOTE:

Es una entidad que nos sirve para relacionar unas tablas con otras.

Una tabla que guarde la relación entre los cursos a los que se ha matriculado cada alumno

id	alumno_id	curso_id
1	5	2
2	3	2
3	2	4

Nos hemos dado cuenta de que la relación curso > alumnos, al ser N to N requería una entidad intermedia llamada matrícula.



FOREIGN KEY CONSTRAINTS

Siempre que tengamos una **llave foránea** deberíamos añadirle unas restricciones para definir qué hacer cuando se **ACTUALICE** un dato o cuando se **BORRE** un dato de la tabla relacionada.

ATRIBUTOS PARA LA FOREIGN KEY:

- **RESTRICT** 🙌 Si intentamos borrar un profesor de la BBDD, **no nos va a dejar** porque sabe que ese profesor está relacionado con un curso.
- **CASCADE** 🙌 Si hay alguna modificación o borrado de un profe, también se va a actualizar el registro del curso o a borrar.
- **SET NULL** 🙌 Si hay algún cambio, se cambiará el valor del campo `profesor_id` a **null** dentro de la tabla `cursos`.
- **SET DEFAULT** 🙌 Si hay algún cambio, se cambiará el valor del campo `profesor_id` a un valor por defecto dentro de la tabla `cursos`.

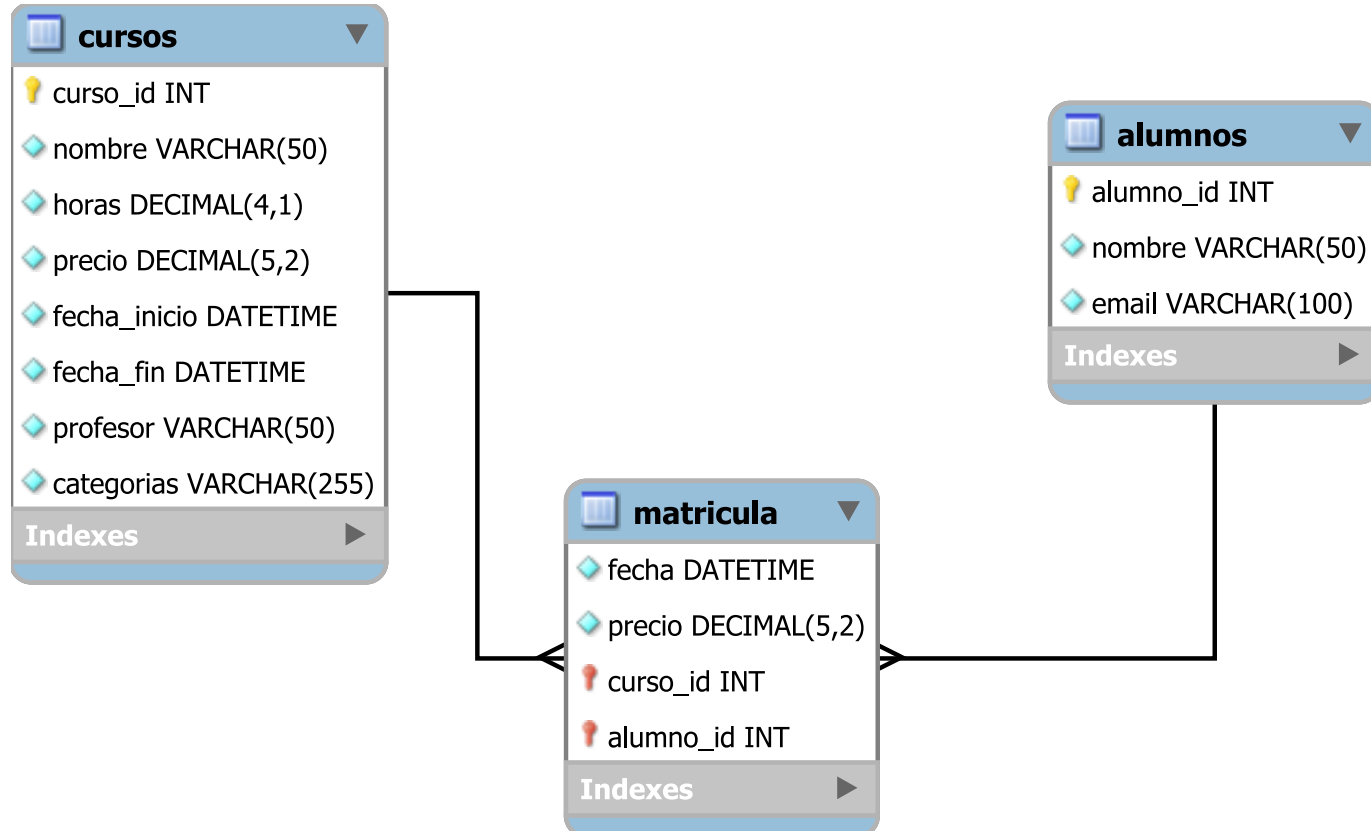
3. MODELO FÍSICO

Una vez terminado, podemos crear nuestro **modelo final**. Que será distinto dependiendo de qué tipo de base de datos usemos.

NORMALIZACIÓN

La normalización son una serie de normas que hacen que nuestras bases de datos estén mucho mejor estructuradas y con muchos menos datos redundantes.

1. PRIMERA FORMA NORMAL 👉 CADA CELDA DEBERÍA TENER UN SOLO VALOR INDIVISIBLE Y NO DEBERÍAMOS TENER COLUMNAS REPETIDAS.



Por ejemplo, si quisiéramos crear una columna dentro de la tabla `cursos` que se llamara `categorias`, sería un error, primero porque habría varias categorías separadas por coma y eso es difícil de trabajar.

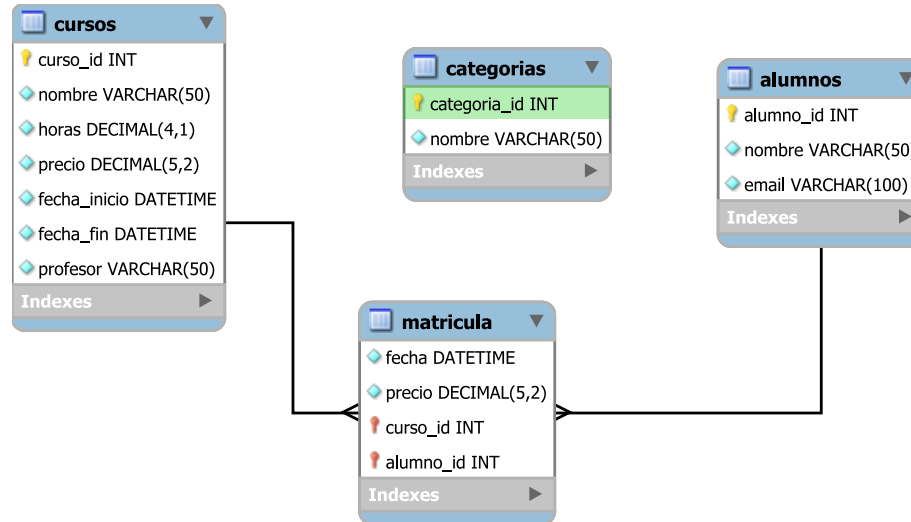
id	nombre	horas	categorias
1	SQL	20	SQL, Python, Java
2	Python	30	Python, SQL
3	Java	40	Java, SQL

✗✗✗

Si, en cambio, quisieramos crear 2 o 3 columnas llamadas `categoria_1`, `categoria_2`, etc. habría columnas repetidas.

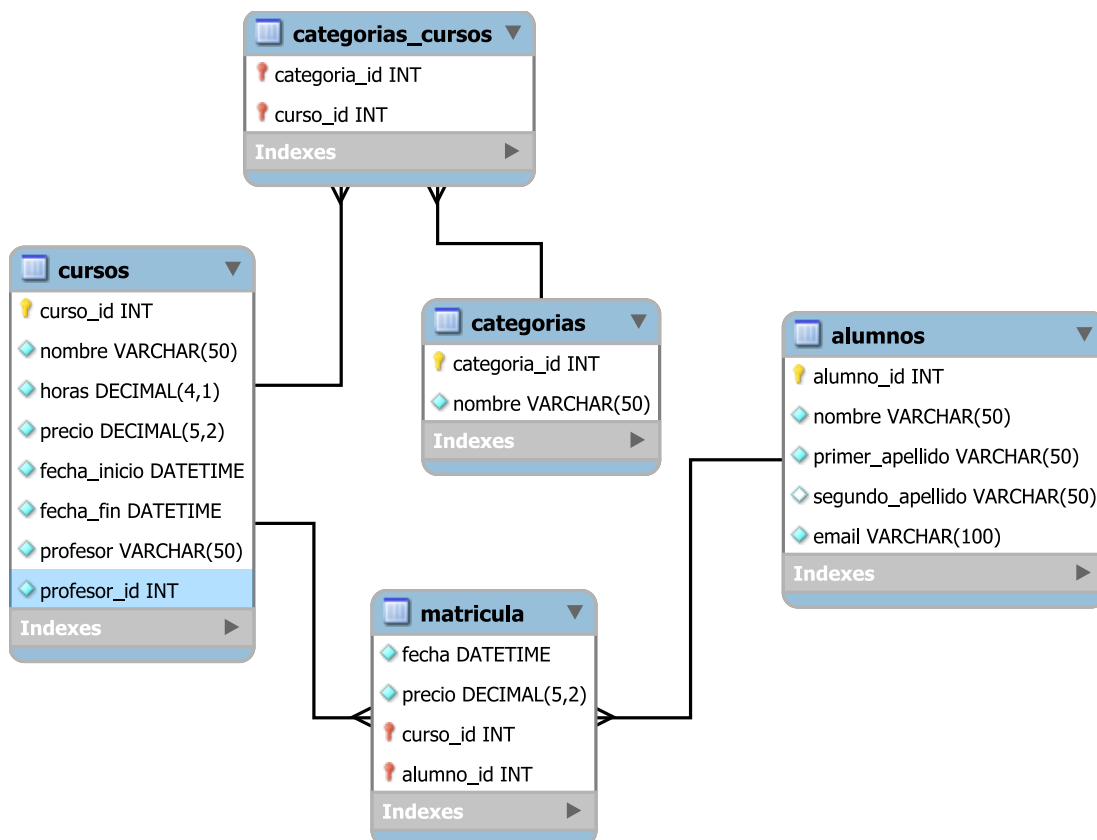
id	nombre	horas	categoria_1	categoria_2	categoria_3
1	SQL	20	SQL	Python	Java
2	Python	30	Python	SQL	
3	Java	40	Java	SQL	

En este caso lo mejor es crear una **tabla de tipo catálogo** llamada `categorias` .

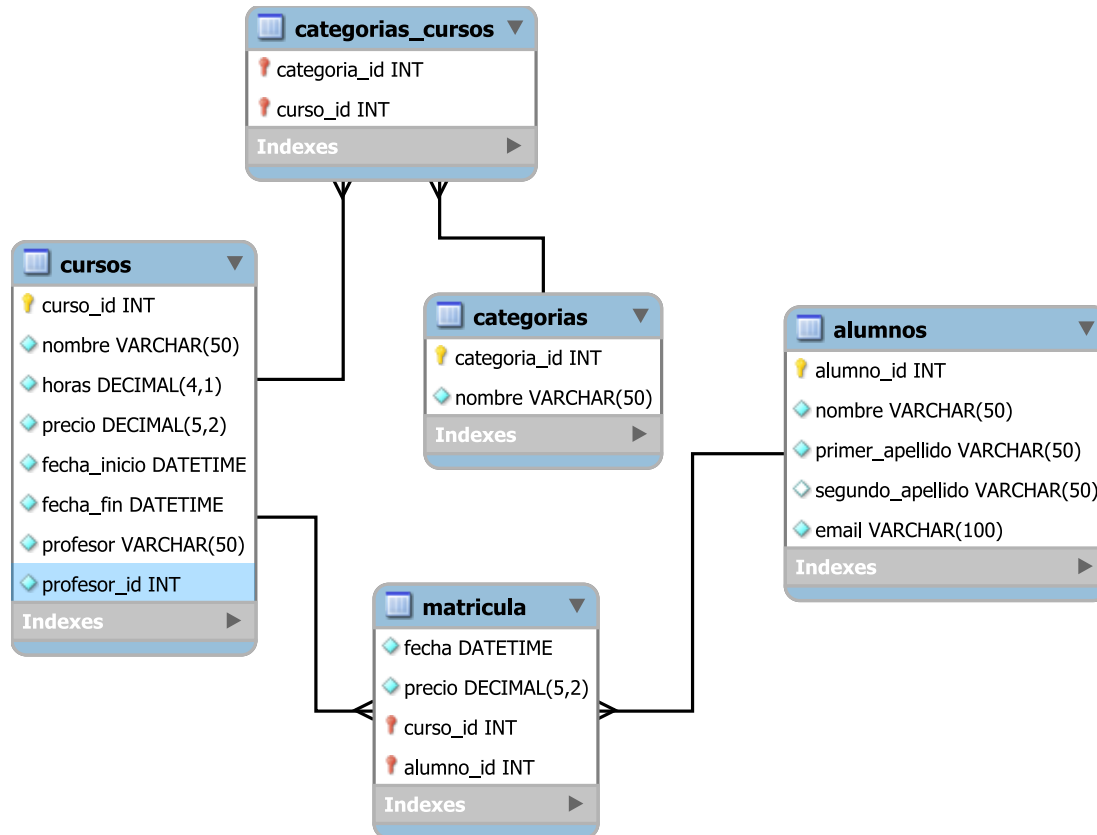


id	categoria
1	SQL
2	Python
3	Java
4	JavaScript

El caso es que esta tabla tiene una relación **“Muchos a Muchos”** con la tabla `cursos` y eso no existe en SQL. En realidad, siempre que pasa esto creamos otra **tabla de enlace** como la de `matriculas`.



Además, también deberíamos separar aquellos campos que puedan ser divididos, como **nombre** en “**nombre**”, “**apellido**” o **dirección** en “**calle**”, “**numero**”, y “**codigo_postal**”.



2. SEGUNDA FORMA NORMAL 🙌 NINGUNA TABLA DEBERÍA TENER CELDAS QUE NO HABLARAN DE “ESA” Y “SOLO ESA” ENTIDAD.

En este caso, si tuviéramos un campo en la tabla `cursos` que se llamara `profesor`, acabaríamos teniendo a lo mejor varios registros con el mismo nombre del profesor porque imparte varios cursos.

Lo suyo sería **sacarlo a otra tabla y relacionarla con esta**. Esto lo veremos siempre cuando veamos **CAMPOS REPETIDOS**.

id	nombre	horas	profesor
1	SQL	20	Ivan
2	Python	30	Ivan
3	Java	40	Pablo
4	JavaScript	50	Ivan

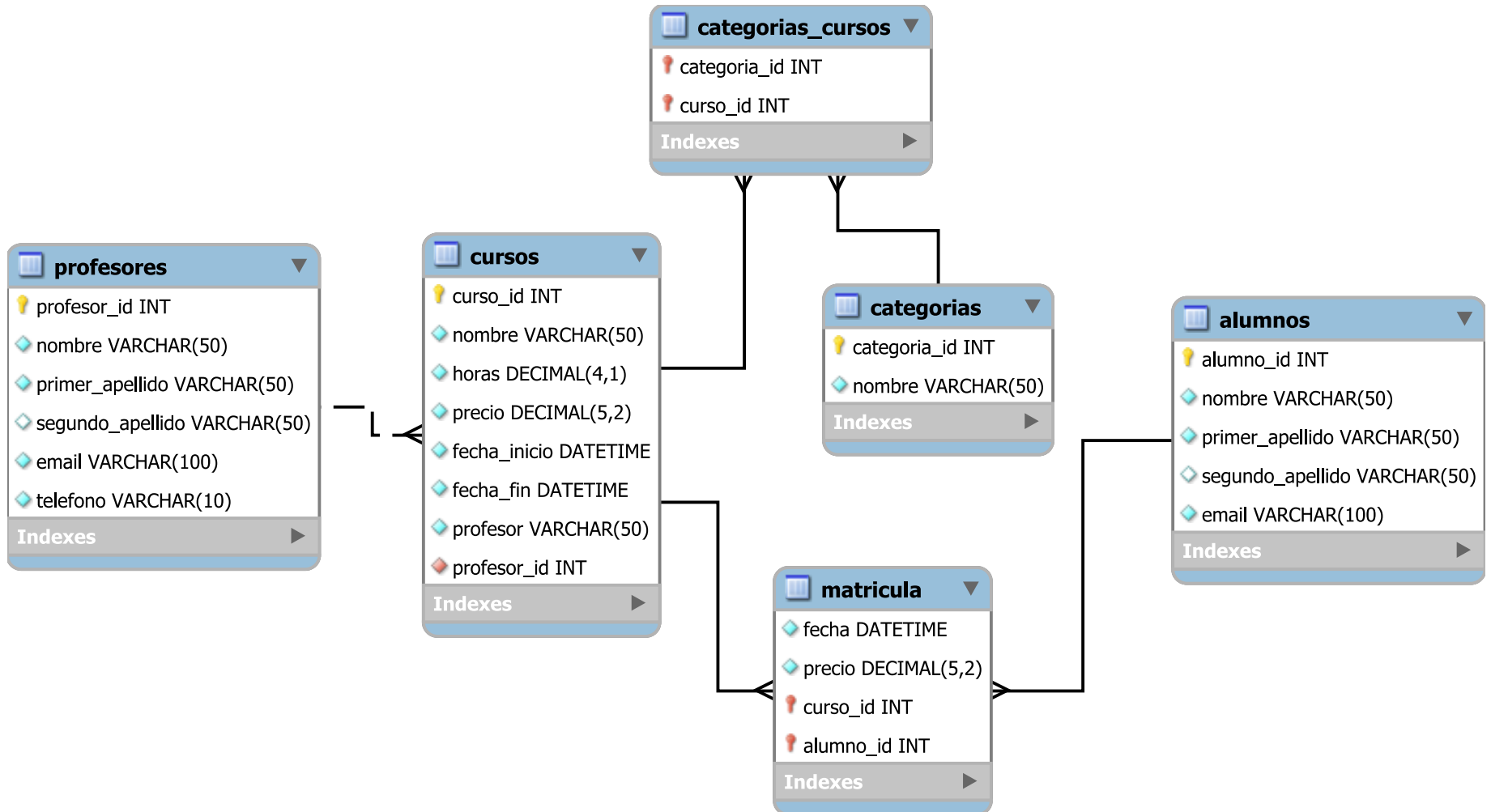
XXX

Cursos			
id	nombre	horas	profesor_id
1	SQL	20	2
2	Python	30	2
3	Java	40	13
4	JavaScript	50	2

↔

Profesores	
id	nombre
2	Ivan
13	Pablo





3. TERCERA FORMA NORMAL 🙌 NINGUNA COLUMNA EN UNA TABLA DEBERÍA SER DERIVADA DE OTRAS COLUMNAS.

Por ejemplo, si tenemos una columna `nombre` y otra columna `apellido`, no deberíamos tener otra columna llamada `nombre_completo` porque esa información ya la podemos sacar de las otras dos.

Este ejemplo también lo vemos cuando tenemos columnas que son cálculos matemáticos de otras columnas.

EN RESUMEN

INTENTA NO TENER CÓDIGO REPETIDO NI DATOS QUE NO FORMEN PARTE DE UNA TABLA.

COMO CONSEJO FINAL
INTENTA NO HACER TABLAS DE
ABSOLUTAMENTE TODO

(solo lo que haga falta para tu modelo de negocio)

EJERCICIO EJEMPLO 01

Se quiere diseñar una Base de Datos para controlar el acceso a las pistas deportivas de Zaragoza. Se tendrán en cuenta los siguientes supuestos:

- Todo aquel que quiera hacer uso de las instalaciones tendrá que registrarse y proporcionar su nombre, apellidos, email, teléfono, dni y fecha de nacimiento.
- Hay varios polideportivos en la ciudad, identificados por nombre, dirección, extensión (en m²).
- En cada polideportivo hay varias pistas de diferentes deportes. De cada pista guardaremos un código que la identifica, el tipo de pista (tenis, fútbol, pádel, etc.), si está operativa o en mantenimiento, el precio y la última vez que se reservó.
- Cada vez que un usuario registrado quiera utilizar una pista tendrá que realizar una reserva previa a través de la web que el ayuntamiento ha creado. De cada reserva queremos registrar la fecha en la que se reserva la pista, la fecha en la que se usará y el precio. Hay que tener en cuenta que todos los jugadores que vayan a hacer uso de la pista deberán estar registrados en el sistema y serán vinculados con la reserva.



